

Feuille de travail — Étude de cas

KANBAN

Concepteur Développeur d'Applications — Niveau 6 — Bloc 1 : Développer une application sécurisée

Nom et prénom du stagiaire : RUKUNDO RONALDO

Dépôt du code (GitHub) : <https://github.com/rronaldo007/jaydee-kanban>

Maquette (Figma) : <https://www.figma.com/design/sfdPYuWhEx89VZgWBdcjwz/Untitled>

Question 1 — Organisation et gestion du projet

Méthode de gestion retenue

J'ai retenu une approche agile de type Scrumban : des sprints courts (Scrum) structurent l'avancement, et un tableau Kanban (colonnes À faire → En cours → En cours de revue → Terminé) rend le flux visible et limite le travail en cours. Ce choix est adapté au contexte : le périmètre se découpe en livrables successifs (conception, initialisation, back-end, front-end, tests, sécurisation), les délais sont stricts et la conception se précise au fil des itérations. Une méthode en cascade aurait été trop rigide pour un périmètre qui s'affine en avançant.

Planification (phases, durées, dépendances)

#	Tâche	Phase	Durée	Dépend de	Échéance
Ex1	Conception et modèle de données	Conception	1 j	—	08/06
Ex2	Maquette fonctionnelle Figma	Interfaces	1 j	Ex1	09/06
Ex3	Initialisation projets Express et React	Environnement	1 j	Ex2	10/06
Ex4	Vérification démarrage et communication	Environnement	1 j	Ex3	11/06
Ex5	Composants métier (colonne et tâche)	Back-end	2 j	Ex3	15/06
Ex6	Routes API et contrôleurs	Back-end	1 j	Ex5	17/06
Ex7	Tests unitaires de l'API	Tests	1 j	Ex6	18/06
Ex8	Interface React du tableau	Front-end	3 j	Ex6	23/06
Ex9	Sécurisation et validation de l'API	Sécurité	2 j	Ex6, Ex7	25/06
—	Recette et validation finale	Validation	1 j	Ex8, Ex9	26/06

Chemin critique : Ex1 → Ex3 → Ex5 → Ex6 → Ex8 → Ex9 → recette. La maquette (Ex2) et les tests (Ex7) se mènent en parallèle sans bloquer ce chemin.

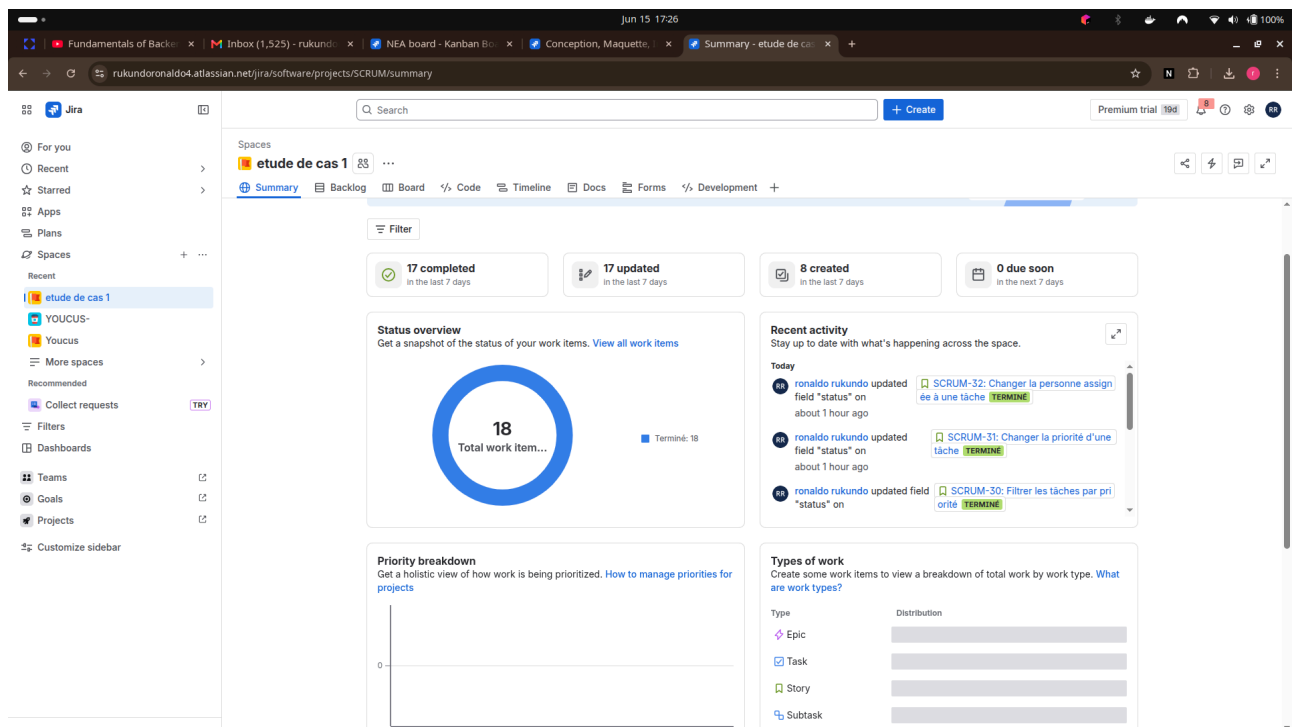
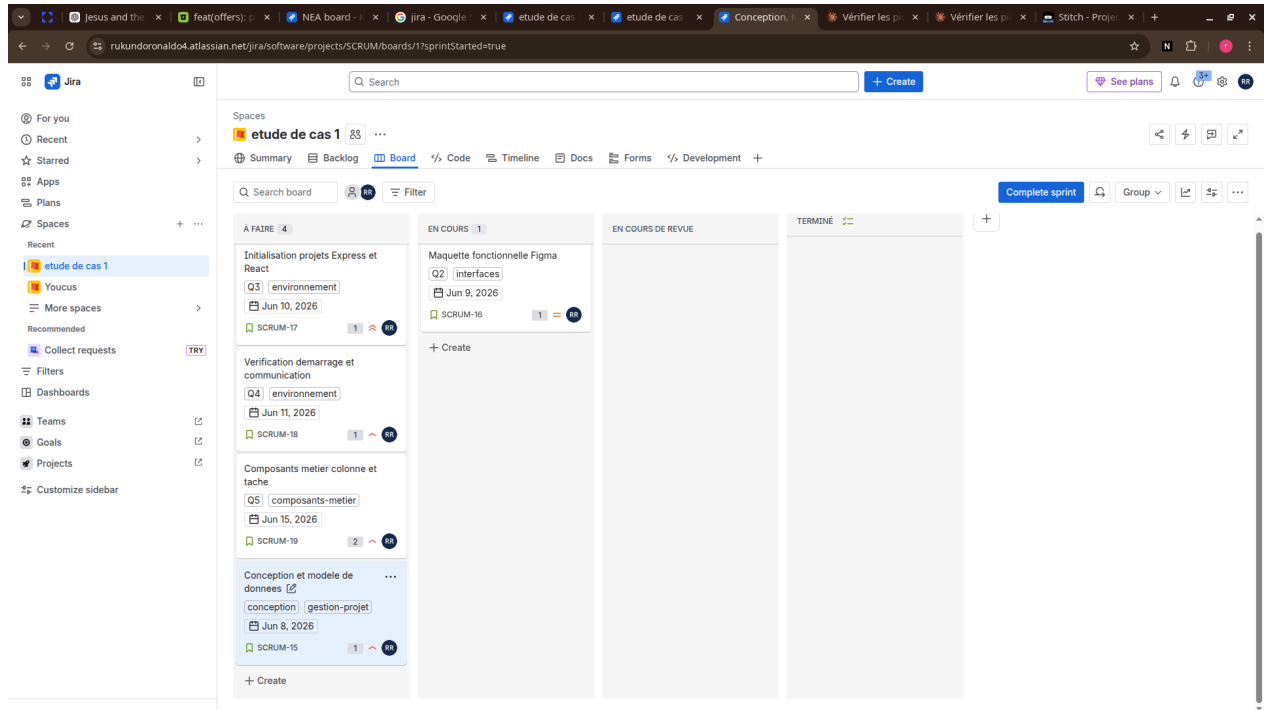
Outils collaboratifs

Besoin	Outil	Usage
Suivi des tâches	Jira (Scrum + Kanban)	stories, sprints datés, colonnes de flux, critères d'acceptation
Versionnement	Git + GitHub	dépôt jaydee-kanban, commits réguliers, historique traçable
Conception UI	Figma	maquette validée avant le développement
Développement	VS Code, Node/Express, React/Vite	environnement cohérent avec l'architecture
Tests / Qualité	Jest + Supertest	exécution via npm test
Démarrage	npm + concurrently	npm start lance les deux serveurs

Suivi, alertes et assurance qualité

- Suivi : mise à jour quotidienne du tableau Jira ; la répartition des cartes donne le reste à faire.
- Alertes : chaque ticket a une échéance ; une carte en retard apparaît en rouge ; revue de fin de sprint.
- Qualité : séparation des responsabilités, tests automatisés, colonne « En cours de revue » avant clôture, validation des données et gestion centralisée des erreurs.

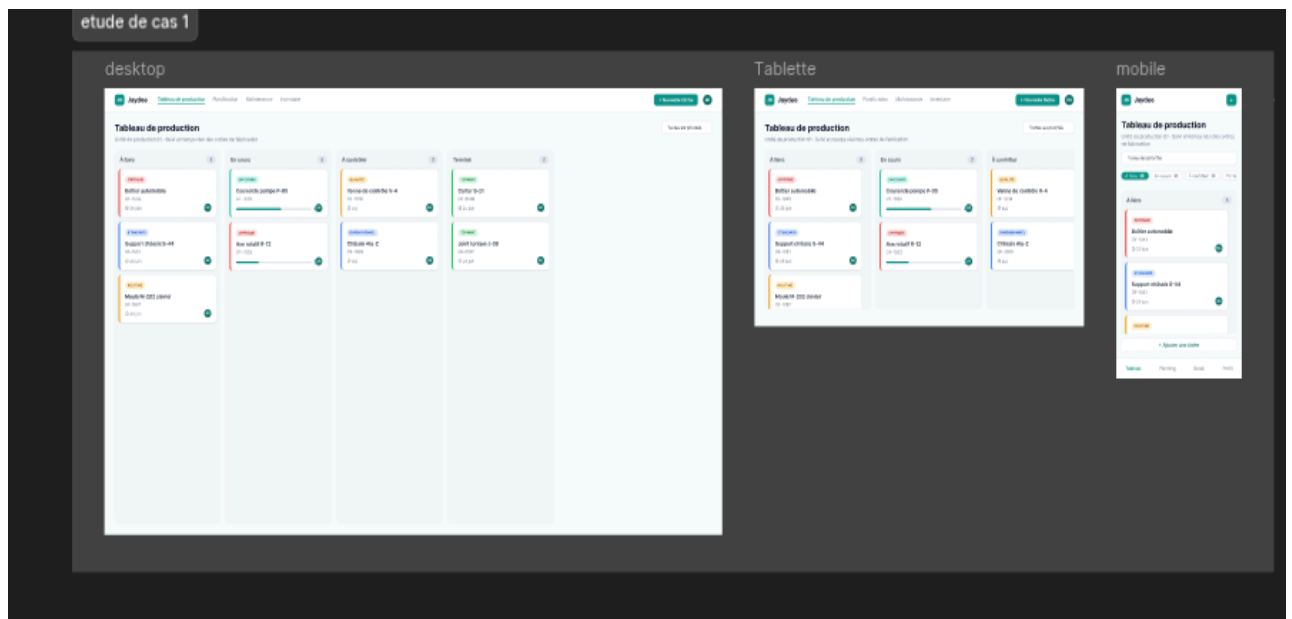
[CAPTURE À INSÉRER : diagramme de Gantt / planning, board Jira, backlog des sprints]



Question 2 — Maquette (conception des interfaces)

La maquette du tableau Kanban a été réalisée sous Figma. Elle représente le flux de production en quatre colonnes (À faire, En cours, À contrôler, Terminé). Chaque tâche est une carte affichant son nom et un repère couleur (badge de priorité). L'interface est pensée pour un usage professionnel : lisibilité, hiérarchie visuelle, cohérence graphique, et adaptation à deux formats (poste fixe et tablette). Le découpage est anticipé en composants réutilisables (colonne, carte de tâche).

[CAPTURE À INSÉRER : maquette Figma (vue desktop et vue tablette)]



Question 3 — Initialisation de l'environnement technique

Deux projets distincts ont été créés : un back-end Node.js + Express et un front-end React (Vite). Les dossiers reflètent une séparation claire des responsabilités : côté serveur routes / contrôleurs / services / modèles / middlewares, côté client des composants réutilisables. Un package.json racine permet de lancer les deux projets ensemble.

backend/package.json

```
{
  "name": "jaydee-kanban-backend",
  "version": "1.0.0",
  "description": "API Express du tableau Kanban interne de l'entreprise Jaydee",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js",
    "test": "jest --runInBand"
  },
  "keywords": ["kanban", "express", "api"],
  "author": "Rukundo Ronaldo",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2"
  },
  "devDependencies": {
    "jest": "^29.7.0",
    "nodemon": "^3.1.4",
    "supertest": "^7.0.0"
  }
}
```

frontend/package.json

```
{
  "name": "jaydee-kanban-frontend",
  "private": true,
  "version": "1.0.0",
  "description": "Interface React du tableau Kanban Jaydee",
  "type": "module",
  "scripts": {
    "start": "vite",
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.1",
    "vite": "^5.4.0"
  }
}
```

Arborescence du projet (commande : `tree -I node_modules`) :

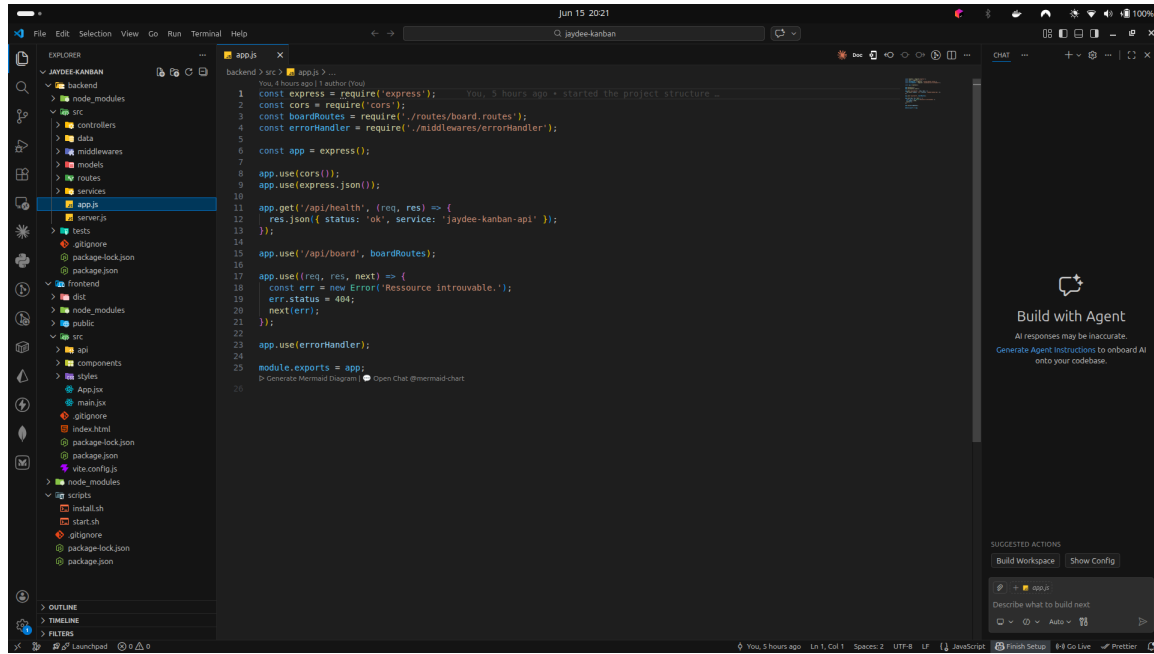
```
jaydee-kanban
├── backend
```

```
├── src
│   ├── controllers
│   ├── data
│   ├── middlewares
│   ├── models
│   ├── routes
│   ├── services
│   ├── app.js
│   └── server.js
├── tests
│   ├── board.routes.test.js
│   └── tasks.routes.test.js
├── package.json
└── package-lock.json

├── frontend
│   ├── public
│   ├── src
│   │   ├── api
│   │   ├── components
│   │   ├── styles
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── index.html
│   ├── package.json
│   ├── package-lock.json
│   └── vite.config.js
├── scripts
│   ├── install.sh
│   └── start.sh
├── package.json
└── package-lock.json
```

17 directories, 16 files

[CAPTURE À INSÉRER : (optionnel) capture d'écran de l'arborescence dans l'éditeur]



Question 4 — Démarrage et communication

Le serveur Express démarre et écoute sur le port 3001 ; l'application React est servie par Vite sur le port 5173. Le proxy Vite redirige les appels /api vers le back-end. La route GET /api/health renvoie un JSON ; l'appeler depuis le front confirme que la chaîne front ↔ API fonctionne.

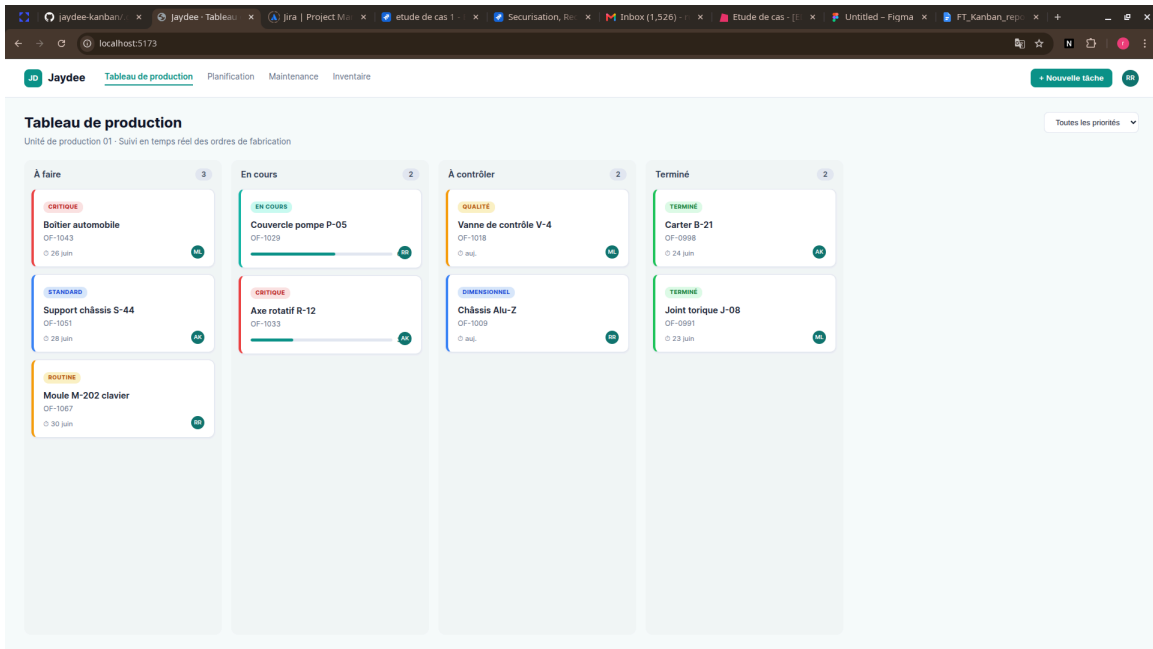
Démarrage du serveur Express (sortie terminal) :

```
API Jaydee Kanban démarrée sur http://localhost:3001
```

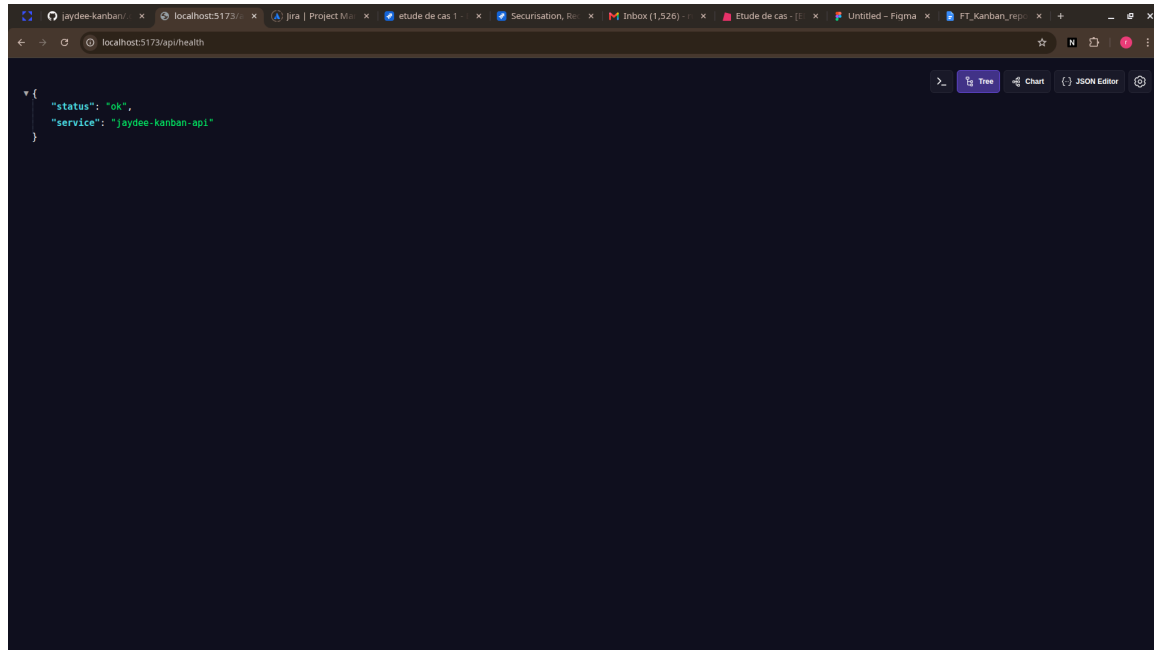
[CAPTURE À INSÉRER : terminal : démarrage des deux serveurs (npm start)]

```
Jun 15 2023
npm start
+ Analyse conversation from Claude chat
+ npm start
+ ronald@ronaldo-Inspiron-7566-2nt:~\Documents\last_year_iscod\etude_de_ca...
- jaydee-kanban git:(main) npm start
> jaydee-kanban@1.0.0 start
> concurrently -n API,WEB -c blue,green "npm --prefix backend run dev" "npm --prefix frontend run dev"
[WEB] > jaydee-kanban-frontend@1.0.0 dev
[WEB] > vite
[API] > jaydee-kanban-backend@1.0.0 dev
[API] > nodemon src/server.js
[API] [nodemon] 3.1.14
[API] [nodemon] to restart at any time, enter `rs`
[API] [nodemon] watching path(s): *.*
[API] [nodemon] watching extensions: js,mjs,cjs,json
[API] [nodemon] starting `node src/server.js`
[WEB] VITE v5.4.21 ready in 326 ms
[WEB] + Local: http://localhost:5173/
[WEB] + Network: use --host to expose
[API] API Jaydee Kanban démarrée sur http://localhost:3001
```

[CAPTURE À INSÉRER : navigateur : application React lancée sur <http://localhost:5173>]



[CAPTURE À INSÉRER : test de communication : <http://localhost:5173/api/health> renvoie le JSON]



Question 5 — Composants métier (côté serveur)

Les modèles Tâche et Colonne sont séparés des données. Une tâche valide possède un identifiant unique, un nom, une couleur hexadécimale et une colonne existante ; chaque règle non respectée lève une erreur. Le module d'initialisation génère 9 tâches de démonstration, toutes validées au démarrage, avec un contrôle d'unicité des identifiants.

models/task.js

```
const HEX_COLOR = /^#[0-9A-Fa-f]{6}$/;

function createTask({ id, name, color, columnId } = {}, validColumnIds = []) {
  if (!Number.isInteger(id) || id <= 0) {
    throw new Error('Tâche invalide : identifiant manquant ou incorrect.');
```

```
  }
  if (typeof name !== 'string' || name.trim() === '') {
    throw new Error('Tâche invalide : nom manquant.');
```

```
  }
  if (typeof color !== 'string' || !HEX_COLOR.test(color)) {
    throw new Error('Tâche invalide : couleur hexadécimale attendue (#RRGGBB).');
```

```
  }
  if (typeof columnId !== 'string' || columnId.trim() === '') {
    throw new Error('Tâche invalide : colonne manquante.');
```

```
  }
  if (validColumnIds.length > 0 && !validColumnIds.includes(columnId)) {
    throw new Error('Tâche invalide : colonne inconnue "${columnId}".');
  }
  return { id, name: name.trim(), color, columnId };
}

module.exports = { createTask };
```

models/column.js

```
function createColumn({ id, name } = {}) {
  if (typeof id !== 'string' || id.trim() === '') {
    throw new Error('Colonne invalide : identifiant manquant.');
```

```
  }
  if (typeof name !== 'string' || name.trim() === '') {
    throw new Error('Colonne invalide : nom manquant.');
```

```
  }
  return { id: id.trim(), name: name.trim() };
}

module.exports = { createColumn };
```

data/seed.js

```
const { createColumn } = require('../models/column');
const { createTask } = require('../models/task');

const columns = [
  createColumn({ id: 'todo', name: 'À faire' }),
  createColumn({ id: 'doing', name: 'En cours' }),
  createColumn({ id: 'review', name: 'À contrôler' }),
  createColumn({ id: 'done', name: 'Terminé' })
];

const columnIds = columns.map((c) => c.id);

const demoTasks = [
```

```

    { id: 1, name: 'Boîtier automobile', color: '#EF4444', columnId: 'todo', priority:
    'CRITIQUE', reference: 'OF-1043', dueDate: '26 juin', assignee: 'ML' },
    { id: 2, name: 'Support châssis S-44', color: '#3B82F6', columnId: 'todo', priority:
    'STANDARD', reference: 'OF-1051', dueDate: '28 juin', assignee: 'AK' },
    { id: 3, name: 'Moule M-202 clavier', color: '#F59E0B', columnId: 'todo', priority:
    'ROUTINE', reference: 'OF-1067', dueDate: '30 juin', assignee: 'RR' },
    { id: 4, name: 'Couvercle pompe P-05', color: '#14B8A6', columnId: 'doing', priority:
    'EN COURS', reference: 'OF-1029', progress: 60, assignee: 'RR' },
    { id: 5, name: 'Axe rotatif R-12', color: '#EF4444', columnId: 'doing', priority:
    'CRITIQUE', reference: 'OF-1033', progress: 30, assignee: 'AK' },
    { id: 6, name: 'Vanne de contrôle V-4', color: '#F59E0B', columnId: 'review',
    priority: 'QUALITÉ', reference: 'OF-1018', dueDate: 'auj.', assignee: 'ML' },
    { id: 7, name: 'Châssis Alu-Z', color: '#3B82F6', columnId: 'review', priority:
    'DIMENSIONNEL', reference: 'OF-1009', dueDate: 'auj.', assignee: 'RR' },
    { id: 8, name: 'Carter B-21', color: '#22C55E', columnId: 'done', priority:
    'TERMINÉ', reference: 'OF-0998', dueDate: '24 juin', assignee: 'AK' },
    { id: 9, name: 'Joint torique J-08', color: '#22C55E', columnId: 'done', priority:
    'TERMINÉ', reference: 'OF-0991', dueDate: '23 juin', assignee: 'ML' }
  ];

  const seenIds = new Set();
  const tasks = demoTasks.map((raw) => {
    if (seenIds.has(raw.id)) {
      throw new Error(`Identifiant de tâche dupliqué : ${raw.id}.`);
    }
    seenIds.add(raw.id);
    const base = createTask(raw, columnIds);
    return {
      ...base,
      priority: raw.priority,
      reference: raw.reference,
      dueDate: raw.dueDate,
      progress: raw.progress,
      assignee: raw.assignee
    };
  });

  module.exports = { columns, tasks };

```

Question 6 — Routes de l'API

La route GET /api/board renvoie, au format JSON et avec un code 200, l'objet { columns, tasks }.
La logique suit la séparation route → contrôleur → service : la route ne réécrit aucune logique métier, elle délègue.

routes/board.routes.js

```
const express = require('express');
const router = express.Router();
const boardController = require('../controllers/board.controller');
const { validateTask } = require('../middlewares/validate');

router.get('/', boardController.getBoard);
router.post('/tasks', validateTask, boardController.createTask);
router.put('/tasks/:id', validateTask, boardController.updateTask);

module.exports = router;
```

controllers/board.controller.js

```
const boardService = require('../services/board.service');
const { badRequest } = require('../middlewares/validate');

exports.getBoard = (req, res) => {
  const board = boardService.getBoard();
  res.status(200).json(board);
};

exports.createTask = (req, res, next) => {
  try {
    const task = boardService.addTask(req.body);
    res.status(201).json(task);
  } catch (e) {
    next(badRequest(e.message));
  }
};

exports.updateTask = (req, res, next) => {
  try {
    const id = Number(req.params.id);
    const task = boardService.updateTask(id, req.body);
    res.status(200).json(task);
  } catch (e) {
    next(e.status ? e : badRequest(e.message));
  }
};
```

services/board.service.js

```
const { columns, tasks } = require('../data/seed');
const { createTask } = require('../models/task');

const DISPLAY_FIELDS = ['priority', 'reference', 'dueDate', 'progress', 'assignee'];

// garde la valeur envoyée si elle existe, sinon celle déjà en place
function withDisplayFields(base, payload, existing = {}) {
  const result = { ...base };
  for (const field of DISPLAY_FIELDS) {
    const value = payload[field] !== undefined ? payload[field] : existing[field];
    if (value !== undefined) result[field] = value;
  }
}
```

```

    return result;
}

function getBoard() {
    return { columns, tasks };
}

function addTask(payload = {}) {
    const { name, color, columnId } = payload;
    const columnIds = columns.map((c) => c.id);
    const nextId = tasks.reduce((max, t) => Math.max(max, t.id), 0) + 1;
    const base = createTask({ id: nextId, name, color, columnId }, columnIds);
    const task = withDisplayFields(base, payload);
    tasks.push(task);
    return task;
}

function updateTask(id, payload = {}) {
    const index = tasks.findIndex((t) => t.id === id);
    if (index === -1) {
        const err = new Error(`Tâche introuvable : ${id}.`);
        err.status = 404;
        throw err;
    }
    const columnIds = columns.map((c) => c.id);
    const { name, color, columnId } = payload;
    const base = createTask({ id, name, color, columnId }, columnIds);
    const updated = withDisplayFields(base, payload, tasks[index]);

    tasks[index] = updated;
    return updated;
}

module.exports = { getBoard, addTask, updateTask };

```

Réponse de GET /api/board (extrait JSON) :

```

{
  "columns": [
    {
      "id": "todo",
      "name": "À faire"
    },
    {
      "id": "doing",
      "name": "En cours"
    },
    {
      "id": "review",
      "name": "À contrôler"
    },
    {
      "id": "done",
      "name": "Terminé"
    }
  ],
  "tasks": [
    {
      "id": 1,
      "name": "Boîtier automobile",
      "color": "#EF4444",
      "columnId": "todo",
      "priority": "CRITIQUE",
      "reference": "OF-1043",
      "dueDate": "26 juin",
      "assignee": "ML"
    },
    {

```

```

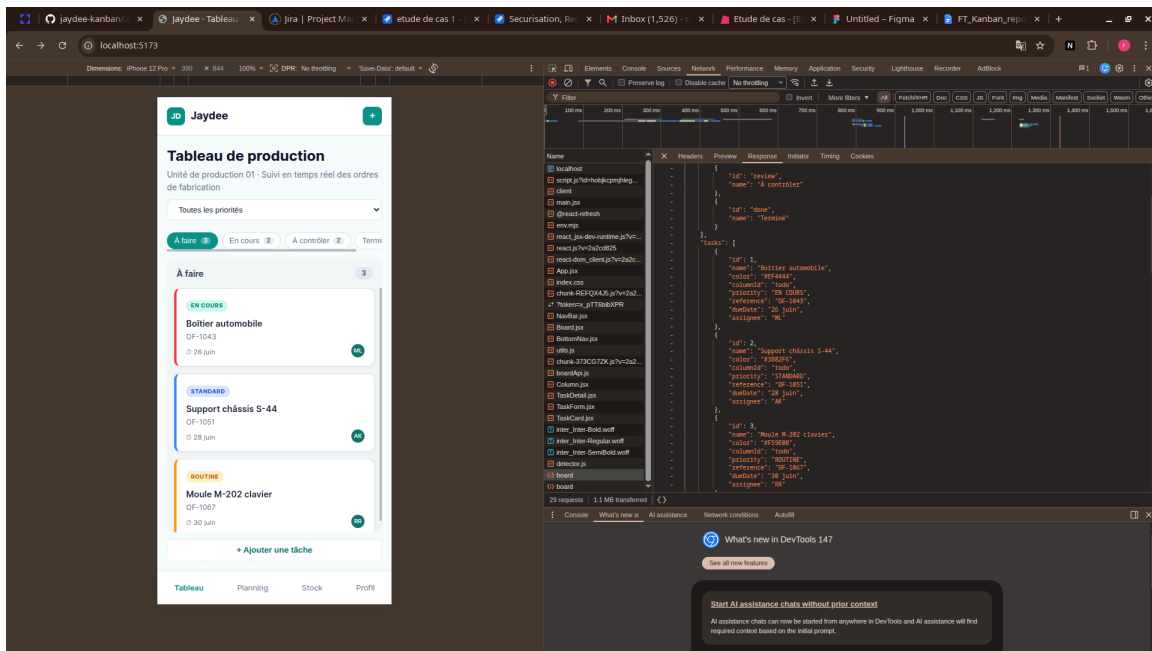
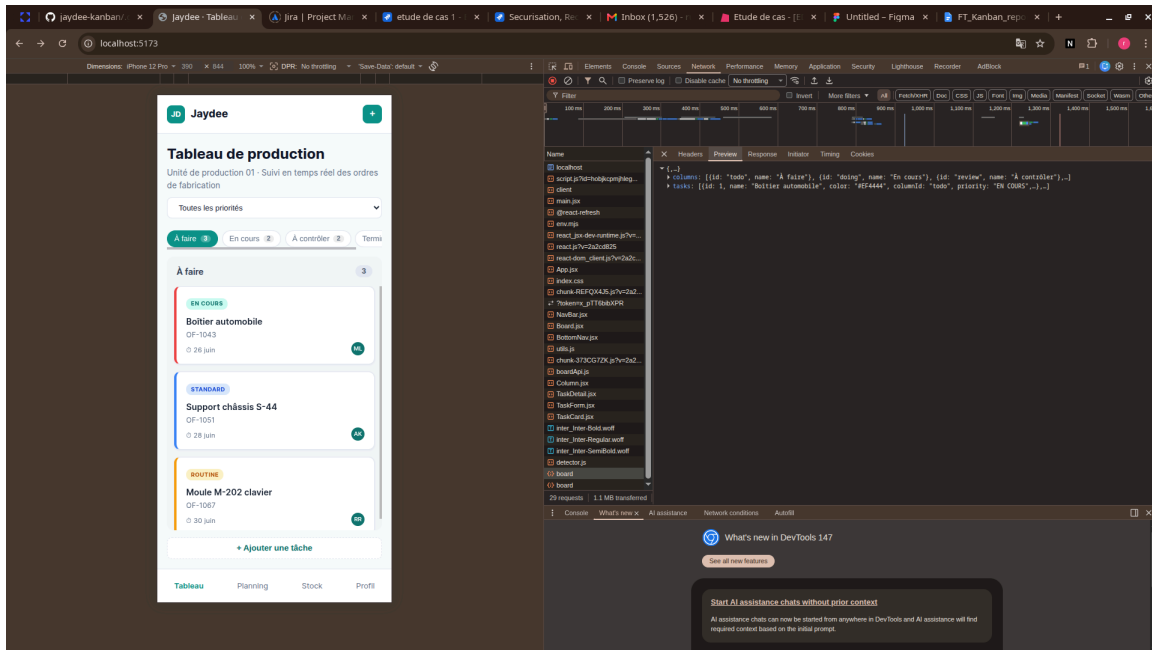
    "id": 2,
    "name": "Support châssis S-44",
    "color": "#3B82F6",
    "columnId": "todo",
    "priority": "STANDARD",
    "reference": "OF-1051",
    "dueDate": "28 juin",
    "assignee": "AK"
  }
}
/* ... 7 autres tâches ... */
}
}

```

[CAPTURE À INSÉRER : (optionnel) réponse JSON de GET /api/board dans le navigateur]

The screenshot displays a web browser window with a Kanban board application running on localhost:5173. The application, titled "Tableau de production", shows a list of tasks in a Kanban format. The tasks are categorized by priority: "EN COURS" (In Progress), "STANDARD", and "ROUTINE". The tasks listed are "Boîtier automobile" (OF-1043), "Support châssis S-44" (OF-1051), and "Moule M-202 clavier" (OF-1067). The board is currently in the "Tableau" (Board) view.

On the right side of the browser, the DevTools network panel is open, showing a GET request to http://localhost:5173/api/board. The request is successful, returning a 200 OK status. The response headers include "Content-Type: application/json" and "Access-Control-Allow-Origin: *". The response body is a JSON array of task objects, which is visible in the "Response" tab.



Question 7 — Tests automatisés

Des tests unitaires (Jest + Supertest) vérifient la route GET /api/board : code HTTP 200, format des données (colonnes et tâches), cohérence des références de colonne, et gestion d'erreur (404 sur une route inconnue). Les tests s'exécutent via npm test.

tests/board.routes.test.js

```
const request = require('supertest');
```

```

const app = require('../src/app');

describe('GET /api/board', () => {
  it('répond avec le code HTTP 200', async () => {
    const res = await request(app).get('/api/board');
    expect(res.status).toBe(200);
  });

  it('retourne du JSON contenant les colonnes et les tâches', async () => {
    const res = await request(app).get('/api/board');
    expect(res.headers['content-type']).toMatch(/application\/json/);
    expect(res.body).toHaveProperty('columns');
    expect(res.body).toHaveProperty('tasks');
    expect(Array.isArray(res.body.columns)).toBe(true);
    expect(Array.isArray(res.body.tasks)).toBe(true);
  });

  it('retourne des tâches au format attendu (id, name, color, columnId)', async () => {
    const res = await request(app).get('/api/board');
    expect(res.body.tasks.length).toBeGreaterThan(0);
    const task = res.body.tasks[0];
    expect(task).toEqual(
      expect.objectContaining({
        id: expect.any(Number),
        name: expect.any(String),
        color: expect.stringMatching(/^#[0-9A-Fa-f]{6}$/),
        columnId: expect.any(String)
      })
    );
    const columnIds = res.body.columns.map((c) => c.id);
    res.body.tasks.forEach((t) => {
      expect(columnIds).toContain(t.columnId);
    });
  });
});

describe('Gestion des erreurs', () => {
  it('retourne 404 sur une route inexistante', async () => {
    const res = await request(app).get('/api/inexistant');
    expect(res.status).toBe(404);
  });
});

```

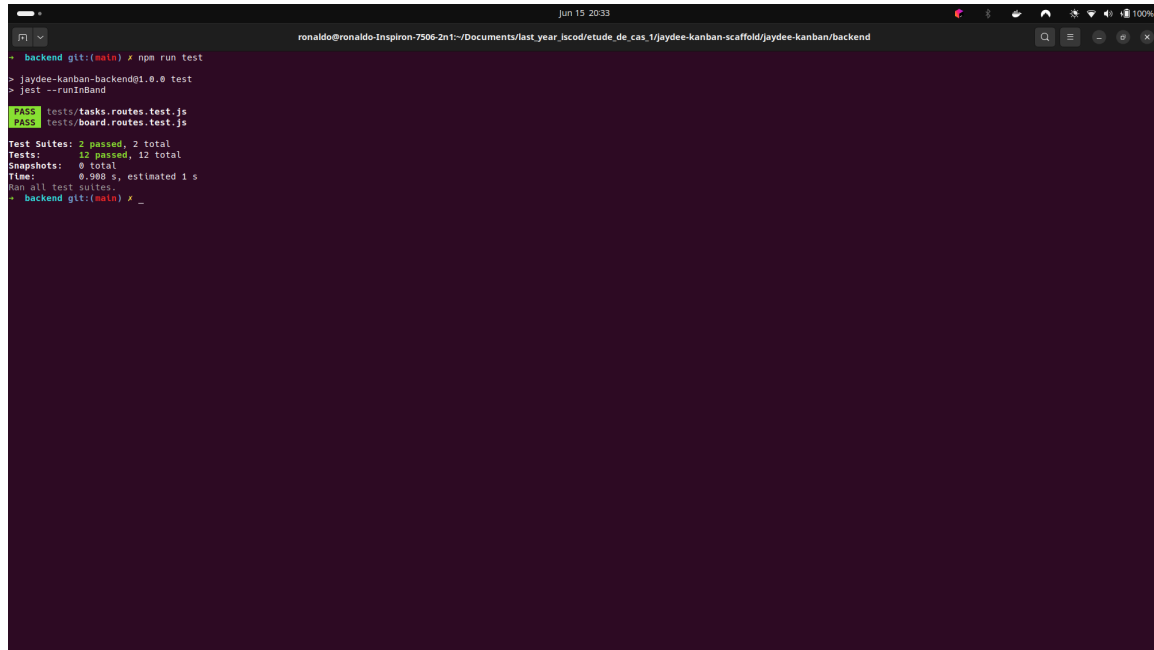
Exécution des tests (npm test) :

```

PASS tests/tasks.routes.test.js
PASS tests/board.routes.test.js
Test Suites: 2 passed, 2 total
Tests:      12 passed, 12 total
Snapshots:  0 total
Time:       0.92 s, estimated 1 s

```

[CAPTURE À INSÉRER : capture d'écran de l'exécution des tests (npm test — 12 verts)]



The screenshot shows a terminal window with the following content:

```
ronaldo@ronaldo-Inspiron-7506-2n1:~/Documents/last_year_iscod/etude_de_cas_1/jaydee-kanban-scaffold/jaydee-kanban/backend
- backend git:(main) ✗ npm run test
> jaydee-kanban-backend@1.0.0 test
> jest --runInBand

PASS tests/tasks.routes.test.js
PASS tests/board.routes.test.js

Test Suites: 2 passed, 2 total
Tests:      12 passed, 12 total
Snapshots:  0 total
Time:       0.988 s, estimated 1 s
Run all test suites.
- backend git:(main) ✗ _
```

Question 8 — Interface utilisateur React

Le composant Board récupère les données via un appel asynchrone et gère trois états : chargement, erreur et données prêtes. L'interface est découpée en composants réutilisables (Board, Column, TaskCard) et reste responsive. Le CSS assure la lisibilité (police, hauteur) et gère le défilement (horizontal entre colonnes, vertical dans une colonne).

src/App.jsx

```
import { useState } from 'react';
import NavBar from '../components/NavBar';
import Board from '../components/Board';
import BottomNav from '../components/BottomNav';
import { PRIORITIES } from '../components/utils';

function App() {
  const [priorityFilter, setPriorityFilter] = useState('all');
  const [showForm, setShowForm] = useState(false);

  return (
    <div className="app">
      <NavBar onNewTask={() => setShowForm(true)} />

      <main className="app__main">
        <div className="app__header">
          <div>
            <h1 className="app__title">Tableau de production</h1>
            <p className="app__subtitle">
              Unité de production 01 · Suivi en temps réel des ordres de fabrication
            </p>
          </div>

          <select
            className="app__filter"
            value={priorityFilter}
            onChange={(e) => setPriorityFilter(e.target.value)}
            aria-label="Filtrer par priorité"
          >
            <option value="all">Toutes les priorités</option>
            {PRIORITIES.map((p) => <option key={p} value={p}>{p}</option>)}
          </select>
        </div>

        <Board
          priorityFilter={priorityFilter}
          showForm={showForm}
          onCloseForm={() => setShowForm(false)}
          onNewTask={() => setShowForm(true)}
        />
      </main>

      <BottomNav />
    </div>
  );
}

export default App;
```

components/Board.jsx

```
import { useEffect, useState } from 'react';
import {
  fetchBoard,
  updateTask as apiUpdateTask,
```

```

    createTask as apiCreateTask
  } from '../api/boardApi';
import { useIsMobile } from './utils';
import Column from './Column';
import TaskDetail from './TaskDetail';
import TaskForm from './TaskForm';

export default function Board({ priorityFilter = 'all', showForm = false, onCloseForm,
onNewTask }) {
  const [columns, setColumns] = useState([]);
  const [tasks, setTasks] = useState([]);
  const [status, setStatus] = useState('loading');
  const [selectedId, setSelectedId] = useState(null);
  const [actionError, setActionError] = useState(null);
  const [activeColumnId, setActiveColumnId] = useState(null);
  const isMobile = useIsMobile();

  useEffect(() => {
    let actif = true;
    fetchBoard()
      .then((data) => {
        if (!actif) return;
        setColumns(data.columns);
        setTasks(data.tasks);
        setActiveColumnId(data.columns[0]?.id ?? null);
        setStatus('ready');
      })
      .catch(() => {
        if (actif) setStatus('error');
      });
    return () => {
      actif = false;
    };
  }, []);

  if (status === 'loading') {
    return <p className="board__message">Chargement du tableau...</p>;
  }

  if (status === 'error') {
    return (
      <p className="board__message board__message--error" role="alert">
        Impossible de charger le tableau. Vérifiez que l'API est démarrée.
      </p>
    );
  }

  const visibleTasks =
    priorityFilter && priorityFilter !== 'all'
      ? tasks.filter((t) => t.priority === priorityFilter)
      : tasks;

  const tasksByColumn = (columnId) => visibleTasks.filter((t) => t.columnId ===
columnId);
  const selectedTask = tasks.find((t) => t.id === selectedId) || null;

  async function applyUpdate(task, changes) {
    const hasChange = Object.keys(changes).some((k) => task[k] !== changes[k]);
    if (!hasChange) return;

    const previous = tasks;
    setActionError(null);
    setTasks((current) =>
      current.map((t) => (t.id === task.id ? { ...t, ...changes } : t))
    );

    try {
      await apiUpdateTask(task.id, {
        name: task.name,
        color: task.color,
        columnId: task.columnId,

```

```

        ...changes
      });
    } catch (e) {
      setTasks(previous);
      setActionError('La mise à jour a échoué. Veuillez réessayer.');
```

```

    }
  }

  async function handleCreate(payload) {
    const created = await apiCreateTask(payload);
    setTasks((current) => [...current, created]);
  }

  const onSelect = (task) => setSelectedId(task.id);

  const overlays = (
    <>
    {selectedTask && (
      <TaskDetail
        key={selectedTask.id}
        task={selectedTask}
        columns={columns}
        onClose={() => setSelectedId(null)}
        onUpdate={applyUpdate}
      />
    )}
    {showForm && <TaskForm columns={columns} onClose={onCloseForm}
    onCreate={handleCreate} />}
  </>
  );

  // Vue mobile : onglets de colonnes + une seule colonne affichée.
  if (isMobile) {
    const activeColumn = columns.find((c) => c.id === activeColumnId) || columns[0];
    return (
      <>
        {actionError && (
          <p className="board__message board__message--error"
            role="alert">{actionError}</p>
          )}
        <div className="board-mobile">
          <div className="col-tabs" role="tablist">
            {columns.map((c) => (
              <button
                key={c.id}
                role="tab"
                className={c.id === activeColumn.id ? 'col-tab col-tab--active' :
'col-tab'}
                onClick={() => setActiveColumnId(c.id)}
              >
                {c.name}
                <span className="col-tab__count">{tasksByColumn(c.id).length}</span>
              </button>
            )}}
          </div>

          {activeColumn && (
            <Column column={activeColumn} tasks={tasksByColumn(activeColumn.id)}
            onSelect={onSelect} />
          )}

          <button type="button" className="mobile-add" onClick={onNewTask}>
            + Ajouter une tâche
          </button>
        </div>
        {overlays}
      </>
    );
  }

  // Vue poste fixe / tablette : colonnes en ligne avec défilement horizontal.
```

```

    return (
      <>
        {actionError && (
          <p className="board__message board__message--error"
            role="alert">{actionError}</p>
        )}
        <div className="board">
          {columns.map((column) => (
            <Column
              key={column.id}
              column={column}
              tasks={tasksByColumn(column.id)}
              onSelect={onSelect}
            />
          ))}
        </div>
        {overlays}
      </>
    );
  }
}

```

components/Column.jsx

```

import TaskCard from './TaskCard';

export default function Column({ column, tasks, onSelect }) {
  return (
    <section className="column" aria-label={column.name}>
      <header className="column__header">
        <h2 className="column__title">{column.name}</h2>
        <span className="column__count">{tasks.length}</span>
      </header>

      <div className="column__tasks">
        {tasks.length === 0 ? (
          <p className="column__empty">Aucune tâche</p>
        ) : (
          tasks.map((task) => (
            <TaskCard key={task.id} task={task} onSelect={onSelect} />
          ))
        )}
      </div>
    </section>
  );
}

```

components/TaskCard.jsx

```

import { priorityStyle, initials } from './utils';

export default function TaskCard({ task, onSelect }) {
  const badge = priorityStyle(task.priority);
  const hasProgress = typeof task.progress === 'number';

  return (
    <article
      className="task-card"
      style={{ borderLeftColor: task.color }}
      role="button"
      tabIndex={0}
      onClick={() => onSelect && onSelect(task)}
      onKeyDown={(e) => {
        if (onSelect && (e.key === 'Enter' || e.key === ' ')) {
          e.preventDefault();
        }
      }}
    >

```

```

        onSelect(task);
    }
    }}
>
{task.priority && (
    <span
        className="task-card__badge"
        style={{ backgroundColor: badge.bg, color: badge.fg }}
    >
        {task.priority}
    </span>
)}

<h3 className="task-card__name">{task.name}</h3>
{task.reference && <p className="task-card__ref">{task.reference}</p>}

{hasProgress ? (
    <div className="task-card__progress">
        <div className="task-card__progress-track">
            <div
                className="task-card__progress-bar"
                style={{ width: `${task.progress}%` }}
            />
        </div>
        <span className="task-card__progress-value">{task.progress}%</span>
    </div>
) : (
    task.dueDate && <p className="task-card__due">⌚ {task.dueDate}</p>
)}

{task.assignee && (
    <span className="task-card__avatar" title={task.assignee}>
        {initials(task.assignee)}
    </span>
)}
</article>
);
}

```

src/styles/index.css

```

:root {
    --teal: #0D9488;
    --teal-dark: #0F766E;
    --ink: #0F172A;
    --slate: #1E293B;
    --muted: #64748B;
    --bg: #F8FAFC;
    --col-bg: #F1F5F9;
    --surface: #FFFFFF;
    --border: #E2E8F0;
    font-family: Inter, system-ui, -apple-system, 'Segoe UI', Roboto, Arial, sans-serif;
    line-height: 1.5;
}

* { box-sizing: border-box; }

html, body, #root { height: 100%; }

body {
    margin: 0;
    background: var(--bg);
    color: var(--slate);
}

/* ---- Layout général ---- */
.app {
    display: flex;

```



```

    flex-direction: column;
    height: 100vh;
}

/* ---- Barre de navigation ---- */
.navbar {
    flex: 0 0 auto;
    height: 64px;
    display: flex;
    align-items: center;
    gap: 2rem;
    padding: 0 2rem;
    background: var(--surface);
    border-bottom: 1px solid var(--border);
}

.navbar__brand { display: flex; align-items: center; gap: 0.6rem; }

.navbar__logo {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    width: 30px;
    height: 30px;
    border-radius: 8px;
    background: var(--teal);
    color: #fff;
    font-size: 0.8rem;
    font-weight: 700;
}

.navbar__title { font-size: 1.1rem; font-weight: 700; color: var(--ink); }

.navbar__menu {
    display: flex;
    gap: 1.5rem;
    list-style: none;
    margin: 0;
    padding: 0;
}

.navbar__link {
    font-size: 0.85rem;
    color: var(--muted);
    cursor: pointer;
    padding-bottom: 2px;
}

.navbar__link--active {
    color: var(--teal);
    font-weight: 600;
    border-bottom: 2px solid var(--teal);
}

.navbar__actions {
    margin-left: auto;
    display: flex;
    align-items: center;
    gap: 1rem;
}

.navbar__new {
    background: var(--teal);
    color: #fff;
    border: none;
    border-radius: 8px;
    padding: 0.5rem 0.9rem;
    font-size: 0.85rem;
    font-weight: 600;
    cursor: pointer;
}

```

```

.navbar__new:hover { background: var(--teal-dark); }

.navbar__avatar {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 28px;
  height: 28px;
  border-radius: 50%;
  background: var(--teal-dark);
  color: #fff;
  font-size: 0.7rem;
  font-weight: 600;
}

/* ---- Zone principale ---- */
.app__main {
  flex: 1 1 auto;
  min-height: 0;
  display: flex;
  flex-direction: column;
  padding: 1.5rem 2rem;
}

.app__header {
  flex: 0 0 auto;
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  margin-bottom: 1.25rem;
}

.app__title { margin: 0; font-size: 1.5rem; font-weight: 700; color: var(--ink); }
.app__subtitle { margin: 0.2rem 0 0; color: var(--muted); font-size: 0.85rem; }

.app__filter {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 0.5rem 0.9rem;
  font-size: 0.8rem;
  color: var(--slate);
  cursor: pointer;
}

/* ---- Tableau Kanban ---- */
.board {
  flex: 1 1 auto;
  min-height: 0;
  display: flex;
  gap: 1rem;
  overflow-x: auto;
  padding-bottom: 0.5rem;
}

/* ---- Colonne ---- */
.column {
  display: flex;
  flex-direction: column;
  flex: 0 0 329px;
  max-height: 100%;
  background: var(--col-bg);
  border-radius: 12px;
  overflow: hidden;
}

.column__header {
  display: flex;
  align-items: center;
  justify-content: space-between;

```

```

padding: 0.85rem 1rem;
flex: 0 0 auto;
}

.column__title { margin: 0; font-size: 0.9rem; font-weight: 600; color: #334155; }

.column__count {
display: inline-flex;
align-items: center;
justify-content: center;
min-width: 28px;
height: 20px;
padding: 0 0.4rem;
background: var(--border);
color: #475569;
border-radius: 10px;
font-size: 0.75rem;
font-weight: 600;
}

.column__tasks {
flex: 1 1 auto;
min-height: 0;
overflow-y: auto;
padding: 0 0.75rem 0.75rem;
display: flex;
flex-direction: column;
gap: 0.75rem;
}

.column__empty {
margin: 0;
color: var(--muted);
font-size: 0.85rem;
font-style: italic;
text-align: center;
padding: 1rem 0;
}

/* ---- Carte de tâche ---- */
.task-card {
position: relative;
background: var(--surface);
border: 1px solid var(--border);
border-left: 4px solid var(--teal);
border-radius: 10px;
padding: 0.85rem 1rem;
box-shadow: 0 1px 2px rgba(15, 23, 42, 0.05);
cursor: pointer;
transition: box-shadow 0.15s ease, transform 0.15s ease;
}

.task-card:hover {
box-shadow: 0 4px 10px rgba(15, 23, 42, 0.1);
transform: translateY(-1px);
}

.task-card:focus-visible {
outline: 2px solid var(--teal);
outline-offset: 2px;
}

.task-card__badge {
display: inline-block;
padding: 0.15rem 0.5rem;
border-radius: 9px;
font-size: 0.6rem;
font-weight: 700;
letter-spacing: 0.5px;
}

```

```

.task-card__name {
  margin: 0.55rem 0 0.15rem;
  font-size: 0.9rem;
  font-weight: 600;
  color: var(--slate);
}

.task-card__ref { margin: 0; font-size: 0.75rem; color: var(--muted); }

.task-card__due { margin: 0.5rem 0 0; font-size: 0.72rem; color: var(--muted); }

.task-card__progress {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  margin-top: 0.6rem;
}

.task-card__progress-track {
  flex: 1 1 auto;
  height: 5px;
  background: var(--border);
  border-radius: 2.5px;
  overflow: hidden;
}

.task-card__progress-bar { height: 100%; background: var(--teal); border-radius: 2.5px; }

.task-card__progress-value { font-size: 0.7rem; font-weight: 600; color: var(--teal-dark); }

.task-card__avatar {
  position: absolute;
  bottom: 0.85rem;
  right: 1rem;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 22px;
  height: 22px;
  border-radius: 50%;
  background: var(--teal-dark);
  color: #fff;
  font-size: 0.6rem;
  font-weight: 600;
}

/* ---- Messages d'état ---- */
.board__message { padding: 1rem; color: var(--muted); }
.board__message--error { color: #B91C1C; font-weight: 600; }

/* ---- Panneau de détail (volet latéral) ---- */
.overlay {
  position: fixed;
  inset: 0;
  background: rgba(15, 23, 42, 0.4);
  display: flex;
  justify-content: flex-end;
  z-index: 50;
}

.detail {
  width: 380px;
  max-width: 90vw;
  height: 100%;
  background: var(--surface);
  box-shadow: -4px 0 20px rgba(15, 23, 42, 0.15);
  padding: 1.5rem;
  overflow-y: auto;
  animation: detail-in 0.15s ease;
}

```

```

}

@keyframes detail-in {
  from { transform: translateX(20px); opacity: 0.6; }
  to { transform: translateX(0); opacity: 1; }
}

.detail__header {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  margin-bottom: 1.25rem;
}

.detail__color {
  width: 12px;
  height: 12px;
  border-radius: 3px;
  flex: 0 0 auto;
}

.detail__title {
  margin: 0;
  font-size: 1.1rem;
  font-weight: 700;
  color: var(--ink);
  flex: 1 1 auto;
}

.detail__close {
  border: none;
  background: transparent;
  font-size: 1.5rem;
  line-height: 1;
  color: var(--muted);
  cursor: pointer;
}

.detail__fields {
  display: grid;
  grid-template-columns: 110px 1fr;
  gap: 0.6rem 1rem;
  margin: 0 0 1.5rem;
}

.detail__fields dt { color: var(--muted); font-size: 0.8rem; }
.detail__fields dd { margin: 0; font-size: 0.85rem; font-weight: 500; }

.detail__edit {
  border-top: 1px solid var(--border);
  padding-top: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.detail__control { display: flex; flex-direction: column; gap: 0.35rem; }
.detail__control > span { font-size: 0.8rem; font-weight: 600; color: var(--slate); }

.detail__control select,
.detail__control input {
  padding: 0.5rem 0.6rem;
  border: 1px solid var(--border);
  border-radius: 8px;
  font-size: 0.85rem;
  background: var(--surface);
  color: var(--slate);
}

.detail__control select { cursor: pointer; }

```

```

/* ---- Formulaire de création ---- */
.form { display: flex; flex-direction: column; gap: 1rem; }

.form__field { display: flex; flex-direction: column; gap: 0.35rem; }
.form__field > span { font-size: 0.8rem; font-weight: 600; color: var(--slate); }

.form__field input,
.form__field select {
  padding: 0.5rem 0.6rem;
  border: 1px solid var(--border);
  border-radius: 8px;
  font-size: 0.85rem;
  color: var(--slate);
  background: var(--surface);
}

.form__colors { display: flex; gap: 0.5rem; }

.form__color {
  width: 26px;
  height: 26px;
  border-radius: 50%;
  border: 2px solid transparent;
  cursor: pointer;
}

.form__color--active { border-color: var(--ink); outline: 2px solid #fff;
outline-offset: -4px; }

.form__error { margin: 0; color: #B91C1C; font-size: 0.8rem; font-weight: 600; }

.form__actions { display: flex; justify-content: flex-end; gap: 0.6rem; margin-top:
0.5rem; }

.form__cancel {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 0.5rem 0.9rem;
  font-size: 0.85rem;
  cursor: pointer;
}

.form__submit {
  background: var(--teal);
  color: #fff;
  border: none;
  border-radius: 8px;
  padding: 0.5rem 0.9rem;
  font-size: 0.85rem;
  font-weight: 600;
  cursor: pointer;
}

.form__submit:disabled { opacity: 0.6; cursor: default; }

/* ---- Bouton « + » compact (mobile) et barre de navigation basse ---- */
.navbar__new-short { display: none; }

.bottom-nav { display: none; }

/* ---- Vue mobile : onglets de colonnes ---- */
.board-mobile {
  flex: 1 1 auto;
  min-height: 0;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.col-tabs {

```

```

display: flex;
gap: 0.4rem;
overflow-x: auto;
padding-bottom: 0.25rem;
}

.col-tab {
display: inline-flex;
align-items: center;
gap: 0.4rem;
white-space: nowrap;
padding: 0.4rem 0.7rem;
border: 1px solid var(--border);
border-radius: 999px;
background: var(--surface);
color: var(--muted);
font-size: 0.8rem;
cursor: pointer;
}

.col-tab--active { background: var(--teal); color: #fff; border-color: var(--teal); }

.col-tab__count {
font-size: 0.7rem;
font-weight: 700;
background: rgba(0, 0, 0, 0.08);
border-radius: 999px;
padding: 0 0.35rem;
}

.col-tab--active .col-tab__count { background: rgba(255, 255, 255, 0.25); }

.mobile-add {
flex: 0 0 auto;
border: 1px dashed var(--border);
background: var(--surface);
color: var(--teal-dark);
border-radius: 8px;
padding: 0.6rem;
font-size: 0.85rem;
font-weight: 600;
cursor: pointer;
}

/* ---- Tablette : on garde le défilement horizontal, colonnes plus étroites ---- */
@media (max-width: 768px) {
.navbar { padding: 0 1rem; gap: 1rem; }
.navbar__menu { display: none; }
.app__main { padding: 1rem; }
.column { flex-basis: 260px; }
}

/* ---- Mobile : layout dédié (onglets + barre basse) ---- */
@media (max-width: 640px) {
.navbar__new-full { display: none; }
.navbar__new-short { display: inline; }
.navbar__new { padding: 0.4rem 0.7rem; font-size: 1rem; }
.navbar__avatar { display: none; }

.app__header { flex-direction: column; gap: 0.6rem; align-items: stretch; }
.app__filter { width: 100%; }

.app__main { padding-bottom: 4.5rem; } /* laisse la place à la barre basse */

.board-mobile .column {
flex: 1 1 auto;
width: 100%;
flex-basis: auto;
max-height: none;
}

```

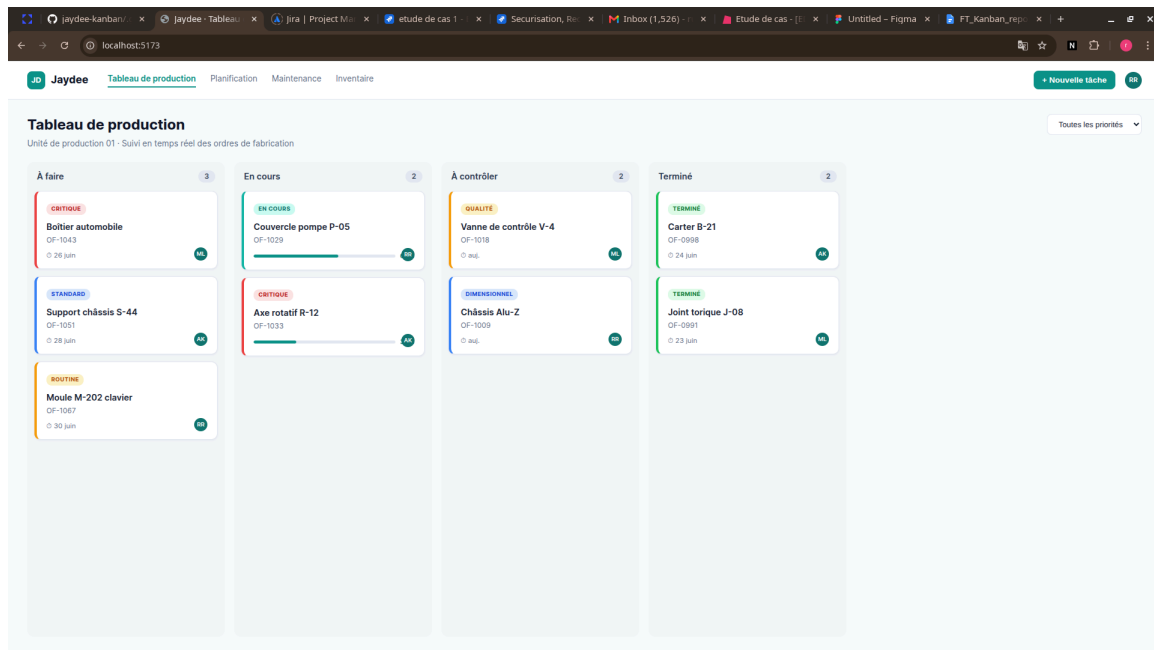
```

.bottom-nav {
  position: fixed;
  bottom: 0; left: 0; right: 0;
  display: flex;
  justify-content: space-around;
  align-items: center;
  height: 3.5rem;
  background: var(--surface);
  border-top: 1px solid var(--border);
  z-index: 40;
}

.bottom-nav__item { font-size: 0.8rem; color: var(--muted); }
.bottom-nav__item--active { color: var(--teal); font-weight: 600; }
}

```

[CAPTURE À INSÉRER : rendu final du tableau dans le navigateur]



Question 9 — Sécurisation et validation de l'API

Un middleware de validation centralisé contrôle toute création ou modification de tâche avant traitement (champs obligatoires, types, couleur hexadécimale) et renvoie une erreur 400 explicite. Un gestionnaire global d'erreurs assure un traitement cohérent et n'expose aucun détail technique (pas de stack) au client. La suite de tests couvre le rejet de données invalides (400) et une création valide (201).

middlewares/validate.js

```
const HEX_COLOR = /^#[0-9A-Fa-f]{6}$/;

function badRequest(message) {
  const err = new Error(message);
  err.status = 400;
  return err;
}

function validateTask(req, res, next) {
  const { name, color, columnId } = req.body || {};
  const errors = [];

  if (typeof name !== 'string' || name.trim() === '') {
    errors.push('le nom est obligatoire');
  }
  if (typeof color !== 'string' || !HEX_COLOR.test(color)) {
    errors.push('la couleur doit être au format hexadécimal #RRGGBB');
  }
  if (typeof columnId !== 'string' || columnId.trim() === '') {
    errors.push('la colonne (columnId) est obligatoire');
  }

  if (errors.length > 0) {
    return next(badRequest(`Données invalides : ${errors.join(', ')}.`));
  }

  next();
}

module.exports = { validateTask, badRequest };
```

middlewares/errorHandler.js

```
// signature à 4 arguments pour qu'Express le voie comme gestionnaire d'erreurs
// eslint-disable-next-line no-unused-vars
module.exports = function errorHandler(err, req, res, next) {
  const status = err.status || 500;
  const message = status < 500 ? err.message : 'Une erreur interne est survenue.';
  res.status(status).json({ error: message });
};
```

src/app.js (branchement du middleware et du gestionnaire)

```
const express = require('express');
const cors = require('cors');
const boardRoutes = require('./routes/board.routes');
const errorHandler = require('./middlewares/errorHandler');

const app = express();
```

```

app.use(cors());
app.use(express.json());

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', service: 'jaydee-kanban-api' });
});

app.use('/api/board', boardRoutes);

app.use((req, res, next) => {
  const err = new Error('Ressource introuvable. ');
  err.status = 404;
  next(err);
});

app.use(errorHandler);

module.exports = app;

```

tests/tasks.routes.test.js (cas de données invalides)

```

const request = require('supertest');
const app = require('../src/app');

describe('POST /api/board/tasks - validation', () => {
  it('rejette une requête sans corps (400)', async () => {
    const res = await request(app).post('/api/board/tasks').send({});
    expect(res.status).toBe(400);
    expect(res.body).toHaveProperty('error');
  });

  it('rejette une couleur invalide (400)', async () => {
    const res = await request(app)
      .post('/api/board/tasks')
      .send({ name: 'Test', color: 'rouge', columnId: 'todo' });
    expect(res.status).toBe(400);
    expect(res.body.error).toMatch(/couleur/i);
  });

  it('rejette une colonne inexistante (400)', async () => {
    const res = await request(app)
      .post('/api/board/tasks')
      .send({ name: 'Test', color: '#FF0000', columnId: 'fantome' });
    expect(res.status).toBe(400);
    expect(res.body).toHaveProperty('error');
  });

  it('crée une tâche valide (201) et la renvoie', async () => {
    const res = await request(app)
      .post('/api/board/tasks')
      .send({ name: 'Nouvelle pièce', color: '#3B82F6', columnId: 'todo' });
    expect(res.status).toBe(201);
    expect(res.body).toEqual(
      expect.objectContaining({
        id: expect.any(Number),
        name: 'Nouvelle pièce',
        color: '#3B82F6',
        columnId: 'todo'
      })
    );
  });

  it('ne divulgue aucune information technique en cas d\'erreur', async () => {
    const res = await request(app).post('/api/board/tasks').send({});
    expect(res.status).toBe(400);
    expect(Object.keys(res.body)).toEqual(['error']);
    expect(res.body).not.toHaveProperty('stack');
    expect(res.body.error).not.toMatch(/\n\s+at\s/);
  });
});

```

```

    });
  });

describe('PUT /api/board/tasks/:id - modification', () => {
  it('modifie une tâche existante (200) et renvoie la version à jour', async () => {
    const res = await request(app)
      .put('/api/board/tasks/1')
      .send({ name: 'Boîtier automobile v2', color: '#22C55E', columnId: 'doing' });
    expect(res.status).toBe(200);
    expect(res.body).toEqual(
      expect.objectContaining({
        id: 1,
        name: 'Boîtier automobile v2',
        color: '#22C55E',
        columnId: 'doing'
      })
    );
  });
});

it('renvoie 404 pour une tâche inexistante', async () => {
  const res = await request(app)
    .put('/api/board/tasks/9999')
    .send({ name: 'Inconnue', color: '#FF0000', columnId: 'todo' });
  expect(res.status).toBe(404);
  expect(res.body).toHaveProperty('error');
});

it('rejette des données invalides (400)', async () => {
  const res = await request(app)
    .put('/api/board/tasks/1')
    .send({ name: '', color: 'bleu', columnId: 'todo' });
  expect(res.status).toBe(400);
  expect(res.body).toHaveProperty('error');
});
});

```

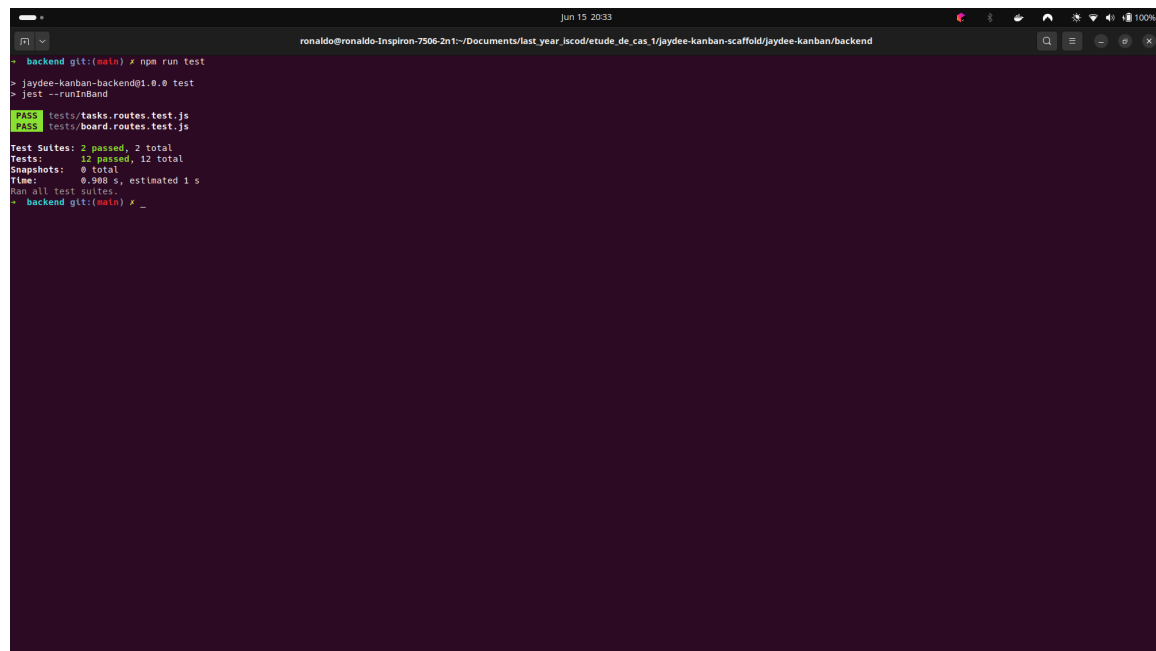
Exécution des tests (npm test) :

```

PASS tests/tasks.routes.test.js
PASS tests/board.routes.test.js
Test Suites: 2 passed, 2 total
Tests:      12 passed, 12 total
Snapshots:  0 total
Time:       0.92 s, estimated 1 s

```

[CAPTURE À INSÉRER : capture d'écran de l'exécution des tests (npm test — 12 verts)]



```
ronaldo@ronaldo-Inspiron-7506-2n1:~/Documents/last_year_iscod/etude_de_cas_1/jaydee-kanban-scaffold/jaydee-kanban/backend
- backend git:(main) ✗ npm run test
> jaydee-kanban-backend@1.0.0 test
> jest --runInBand

PASS tests/tasks.routes.test.js
PASS tests/board.routes.test.js

Test Suites: 2 passed, 2 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Time: 0.988 s, estimated 1 s
Run all test suites.
- backend git:(main) ✗ _
```